

POLER-OS

Технический архитектурный аудит

Версия 3.0 / Июль 2026

Файловая система POLER-FS / Сравнение с 12 ФС / Юридический анализ / Совместимость Windows / Контейнерная изоляция процессов / Криптография как инструмент быстрого действия / Античит как гость / Roadmap / Критический аудит

Обновления v3.0: Полный пересмотр концепции криптографии — от «защиты» к «быстродействию и совместимости». Введена модель криптологики vs бинарной логики (аналогия с клавиатурным вводом). Античит определён как гость в изолированном контейнере. Уточнена контейнерная модель: один контейнер = одно ПО, невозможность межпроцессного взаимодействия. Убрана рекомендация о замене собственной криптографии.

1. POLER-FS: Технический аудит файловой системы

Данный раздел содержит глубокий технический аудит предлагаемой файловой системы POLER-FS. Анализ критически оценивает каждое архитектурное решение и предлагает улучшения там, где текущий дизайн не соответствует современным требованиям. POLER-FS является ключевым компонентом архитектуры POLER-OS, обеспечивая NT-совместимую модель ACL, Alternate Data Streams, reparse points и другие механизмы, необходимые для запуска Windows-приложений нативно. Файловая система спроектирована с учётом NVMe-first подхода, где приоритет отдаётся минимальной латентности и максимальной пропускной способности, а не совместимости с устаревшими вращающимися носителями.

1.1 Общая архитектура: Block Groups vs Extent-based

Предложенная архитектура block groups заимствована из ext2/3/4. Для ФС, заявленной как современная и NVMe-first, это фундаментально ошибочный выбор. Block groups создают фиксированное число inode в группе, приводят к фрагментации, требуют $O(n)$ сканирования bitmap и не масштабируются до 100TB+ томов. Рекомендуемая альтернатива — Extent-based архитектура с B+ деревом. Каждый файл описывается набором extent-ов (логический диапазон файлов, физический диапазон на диске, длина, флаги). Extent-ы хранятся в B+ дереве внутри inode (inline root) и при переполнении — в отдельных блоках. Это даёт: минимум метаданных для больших файлов, естественную поддержку sparse files, совместимость с CoW и NVMe range I/O.

Критерий	Block-based (предложено)	Extent-based (рекомендуется)
Фрагментация метаданных	Высокая	Низкая
Метаданные для 100GB файла	~200KB indirect blocks	~4KB extent tree
Random read	2-3 IOPS на блок	1 IOPS (extent-LBA)
Sparse files	Требует спец. флагов	Естественная поддержка
CoC совместимость	Плохая	Отличная
NVMe оптимизация	Ограниченная	Прямая (extent-range)

POLER-FS ДОЛЖЕН быть extent-based. Block-based подход безнадёжно устарел. Ext4 сама перешла на extents в 2008 году.

1.2 Суперблок

Один 4KB суперблок без резервных копий — критически недостаточно. Предложенный дизайн не содержит UUID тома, feature flags, версии on-disk формата, контрольных сумм. Без этих полей ФС нежизнеспособна в продакшене. Рекомендуемый суперблок должен содержать: 128-bit UUID тома, метку тома (UTF-16LE для NT compat), геометрию (block/sector/page/cluster size), позицию и размер inode таблицы (как extent map), позицию и размер журнала, feature flags (compat/ro_compat/incompat/special), Wrapped КЕК для шифрования, алгоритм контрольной суммы, BLAKE3 хеш суперблока. Резервные копии: минимум 5 копий — блок 0 (primary), блок 1 (backup #1), N/1024 (промежуточные), конец тома (backup #4-5). При загрузке — читаем primary, проверяем checksum, при failed — сканируем все копии.

1.3 Организация Inode

256 байт — критически мало для inode, который должен содержать NT ACL, шифрование, integrity-поля и NVMe-оптимизации. После размещения всех необходимых полей не остаётся места для inline data, project ID, FILE_ATTRIBUTE_*, Alternate Data Streams, repase point data. Рекомендуемый размер: 512 байт. Это позволяет: inline data до ~108B (для файлов до 108 байт — нулевая фрагментация), inline ACL для простых случаев (24B), inline Owner/Group SID (по 16B), B+ tree root для extent map (96B inline), поля шифрования (algo + key_id + nonce + version = 40B), Merkle root hash (32B), NVMe optimization fields, ADS count + ADS root. Direct/indirect pointers категорически недопустимы. Только extent-based B+ дерево.

1.4 Журналирование

Предложен WAL (Write-Ahead Logging) — правильный базис. Но нужны уточнения. Рекомендуется гибридный подход: метаданные — всегда через WAL (журналируем все мутации), данные — CoW для файлов с флагом CoW + Ordered для остальных. Три режима журналирования: writeback (metadata only), ordered (default, data before metadata commit), journal_all (full ACID, для реестра и критичных файлов). Размер журнала: автоматический = $\min(\text{volume_size} / 256, 4\text{GB})$, минимум 32MB для SSD/NVMe, рекомендовано 256MB-1GB.

1.5 Размер блока

4KB как единственный размер — субоптимален. Рекомендуется двухуровневая система: block_size=4KB (физический, атомарная запись, совместим с x86 page) + cluster_size (единица выделения, переменный: 4KB/16KB/64KB). На NVMe физический page = 16-32KB, erase block = 4-16MB, оптимальный write align = 64-128KB. Решение: cluster_size = 16KB (default для NVMe), write combining для группировки 4KB writes, NVMe Write Zeroes для sparse extent-ов, TRIM/DISCARD на границах erase blocks.

1.6 Масштабируемость и NVMe-производительность

Block groups не масштабируются до 100TB+. Том 100TB: 100,000 групп, 3.2GB только bitmap overhead, fsck = часы. С extent-based: Allocation B+ tree ($O(\log N)$ поиск), динамические inode-ы (не привязаны к группе), параллельный fsck по subtree-ам. Для миллиарда файлов: inode таблица как B+ дерево (не массив!), ACL deduplication (большинство ACL идентичны), SID-inode lookup через Bloom filter + hash table.

Per-CPU submission queues (2 на CPU: read/write separation), completion queues (1 на CPU, polling mode), queue depth 256-1024. IO_uring-подобный интерфейс: submission ring + completion ring, inode-based submission (нет path-lookup-open), registered buffers (zero-copy), linked operations. Zero-copy: O_DIRECT + registered buffers для DMA, mmap() с MAP_SHARED, sendfile()/splice() эквиваленты. Write combining buffer для группировки 4KB в 16KB aligned writes. Целевой WAF: sequential 1.1-1.3, random 1.5-2.5.

1.7 HDD-производительность, большие и мелкие файлы

Предложенный дизайн не упоминает HDD вообще — это ошибка. Внешние HDD распространены для backup/archive. Решения: rotational awareness flag в superblock (auto-detect через NVMe IDENTIFY vs SCSI INQUIRY), seek-optimized placement (cylinder groups), aggressive readahead (1MB+), larger cluster_size (64KB), directory B+ tree ($O(\log N)$ seeks), journal на начале диска.

Extent-based подход идеально подходит для файлов 100GB+. Contiguous file = 1 extent (16B inline), moderately fragmented (50 extents) = 800B inline, heavily fragmented (10K extents) = B+ tree depth=1 (~30 leaf nodes). Нужны: preallocation API (PREALLOC_ZEROED, PREALLOC_UNINIT для SetFileValidData

compat), native sparse file support через extent map holes.

Inline data обязателен для мелких файлов. Без inline data: 10М файлов по 1KB = 45GB (67% waste). С inline data: 5GB (9x меньше, 2x меньше IOPS). Directory B+ tree index: SipHash-2-4 (DoS-resistant), ~170 entries per 4KB leaf, depth=1 handles ~29К файлов, depth=2 ~5М файлов.

1.8 Снапшоты, CoW, дедупликация

Per-filesystem CoW — ошибочная архитектура (проблема ZFS/Btrfs). POLER-FS должен использовать per-file CoW с гибким переключением. CoW OFF (default): in-place write + journal protection — для БД, логов, временных файлов. CoW ON: автоматически для VM disk images, snapshotted directories, registry hives. Extent-level CoW (рекомендуемая гранулярность), refcount tree (только shared blocks, refcount>1), интеграция с VSS API Windows.

Offline dedup как primary, optional inline для VM workloads. Inline dedup добавляет неприемлемую latency (~50us на NVMe budget ~100us). Offline: Phase 1 — сканирование с BLAKE3 hash + Cuckoo filter (~3GB на 1TB), Phase 2 — batched dedup с journal protection. Per-directory opt-in для inline dedup.

1.9 Шифрование и контроль целостности

Два уровня шифрования: (1) Per-volume (BitLocker equivalent) — AES-256-XTS, transparent для ФС, HW acceleration AES-NI. (2) Per-inode (EFS equivalent) — AES-256-XTS для disk, AES-256-GCM для новых файлов, ChaCha20-Poly1305 как fallback (нет HW AES). Filename encryption: AES-256-CTS. Key hierarchy: Master Key (TPM 2.0 sealed) -> Volume Key -> File Key (HKDF per-inode). Данные пользователя защищены стандартными алгоритмами — это принципиально отличается от внутренней криптографии ОС, которая рассматривается в разделе 6.

Per-inode SHA-256 integrity — недостаточен. Нужен: per-block checksum (BLAKE3-128 truncated, 16B в block trailer) + Merkle tree (BLAKE3-256, root hash в inode). BLAKE3 в 3-5x быстрее SHA-256, имеет native Merkle tree support, поддерживает keyed mode (MAC). SHA-256 как fallback для FIPS compliance. Block layout: Data (4072B) + Checksum (24B) = 4096B. Scrub: фоновый traversal B+ tree с BLAKE3 верификацией.

1.10 NT-совместимые ACL и совместимость с Windows API

NT ACL — правильный выбор, но реализация требует: ACL deduplication (большинство файлов имеют идентичный наследуемый ACL, храним один раз по ссылке по ID), SACL support (audit ACL — критически важен для Active Directory, compliance, anti-cheat), ACE inheritance (4 флага: OI, CI, IO, NP), ACL cache (hash table: acl_id -> parsed ACL, LRU eviction, per-CPU RCU-protected lookup), VFS translation layer (NT ACL для POLER-FS, POSIX ACL для ext2, None для FAT32).

Самый критичный аспект POLER-FS — совместимость с Windows API. Предложенный дизайн должен обязательно поддерживать: Alternate Data Streams (ADS) — множественные \$DATA атрибуты с именами, каждый ADS — полный файл с ACL, size, extent map; Reparse Points — символические ссылки, junction points, mount points, UWP APPEXECLINK; Mandatory file locking (не advisory как POSIX!); Oplocks (Opportunistic Locks) — критичны для SMB/local anti-cheat мониторинга.

1.11 Восстановление после сбоя, кэширование, VFS

Целевое время восстановления: NVMe <200ms, HDD <5s. Оптимизации: checkpointing (не replay весь журнал, только после последнего checkpoint, интервал 5-30 сек), parallel replay (multi-queue NVMe), lazy orphan cleanup (mark + background free), soft recovery mode (mount RO сразу, replay в фоне).

Multi-level кэширование: L1 CPU caches (inode hot fields, dentry cache, ACL parsed), L2 Page Cache (inode cache 2% RAM, dentry cache 1%, page cache 40-60%), L3 NVMe. Write policy: file data — write-back + ordered flush, metadata — write-through via journal, encryption keys — write-through + zeroize on evict. Dirty page flush: 10% RAM -> background flush, 20% -> aggressive, 40% -> all blocked, dirty age >30s -> flush.

VFS abstraction layer: NT API -> I/O Manager (IRP routing, Fast I/O path, async I/O) -> VFS -> filesystem drivers. VFS operations: mount/umount, lookup, create/unlink, read/write, readdir, ioctl, mmap, truncate, lock. Mount points, union mounts. VFS ACL translation: POLER-FS (NT ACL) <-> VFS ACL <-> ext2 (POSIX ACL) / FAT32 (None->Everyone:Full).

2. Сравнение POLER-FS с 12 файловыми системами

Ниже приведён сравнительный анализ POLER-FS с 12 существующими файловыми системами. Для каждой указаны сильные и слабые стороны, что стоит перенять и что категорически не стоит повторять. Данный анализ подтверждает, что POLER-FS занимает уникальную нишу: собственная ФС с NT-совместимой моделью ACL, спроектированная с нуля для NVMe-first работы и встроенной поддержки Windows-механизмов (ADS, reparse points, oplocks).

ФС	Сильные стороны	Слабые стороны	Что перенять	Не повторять
FAT32	Универсальная совместимость; минимальная сложность	4GB предел; фрагментация; нет целостности	Самодостаточный superblock	Отсутствие журналирования; линейный поиск
NTFS	MFT; ADS; Reparse Points; NT ACL; USN Journal	Фрагментация MFT; нет CoW; нет checksumming	Атрибутную модель MFT; NT ACL; USN Journal	In-place overwrite без CoW
ext4	Extent-based; delayed allocation; inline data	Нет CoW; нет integrity; 16TB предел файла	Delayed allocation + mballoc; inline data	Отсутствие end-to-end integrity
XFS	Allocation Groups; B+ дерево; reflink; online fsck	Невозможно shrink; serial WAL	AG для NVMe queue; reverse mapping; online fsck	Serial WAL при параллельных AG
Btrfs	CoW-дерево; per-file CoW; checksumming	CoW = фрагментация для БД; RAID5/6 ненадёжны	CoW + WAL; per-extent checksumming; subvolumes	CoW-индуцированную фрагментацию
ZFS	End-to-end integrity; CoW; RAID-Z; encryption	Memory hunger (50% RAM); write amplification 3-5x	Merkle tree; ZIL; scrub; native encryption	Memory hunger; неконтролируемую CoW

Сводная таблица сравнения

ФС	Аллокация	Журнал	CoW	Шифр.	Integ.	ACL	Снапш.
FAT32	Цепочка	Нет	Нет	Нет	Нет	Нет	Нет

ФС	Аллокация	Журнал	CoW	Шифр.	Integ.	ACL	Снапш.
NTFS	Экстенты	WAL	Нет	EFS	Нет	NT	VSS
ext4	Экстенты	WAL	Нет	fsencrypt	Нет	POSIX	Нет
XFS	B+tree	WAL	Reflink	Нет	Нет	POSIX	Reflink
Btrfs	CoW tree	CoW	Да	Нет	CRC32c	POSIX	O(1)
ZFS	CoW rec.	CoW+ZIL	Да	AES-GCM	Merkle	NFSv4	O(1)
APFS	CoW tree	CoW	Да	AES-XTS	Нет	POSIX	O(1)
POLER-FS	B+tree ext	WAL	Per-file	AES-XTS	BLAKE3	NT	CoW

3. Юридический анализ проекта

3.1 Совместимость API — законна ли реализация Windows API?

Да, законна. Ключевые прецеденты: Oracle v. Google (2021) — Верховный суд США постановил, что API интерфейсы являются методом операции (method of operation), и их копирование для обеспечения совместимости constitutes fair use. Это прямой прецедент для POLER-OS. Lotus v. Borland (1996) — суд постановил, что интерфейсные элементы (имена, сигнатуры, меню) не охраноспособны авторским правом. Sony v. Connectix (2000) — эмуляция коммерческого продукта законна, даже если требует reverse engineering. Директива ЕС 2009/24/EC, Статья 6 — прямо разрешает декомпиляцию и reverse engineering для обеспечения совместимости, если информация не была ранее доступна иными средствами.

3.2 Clean-room разработка и риски

Clean-room design — юридически признанный метод, при котором команда спецификации отделена от команды реализации. В случае одного разработчика рекомендуется модифицированная чистая комната: документирование каждого решения через black-box анализ (вход-выход), фиксация sources of information для каждого API, запрет на использование leak-исходников, использование только публичной документации (MSDN, Windows SDK headers, ReactOS wiki).

Тип риска	Уровень	Описание	Митигация
Copyright (код)	Низкий	Копирование исходного кода Windows	Clean-room: собственный код с нуля
Патенты (ФС)	Средний	Патенты Microsoft на NTFS/ReFS	POLER-FS вместо NTFS; свой формат
Патенты (API)	Низкий	Патенты на конкретные алгоритмы	Альтернативные реализации поведения
Товарные знаки	Низкий	Использование Windows	Не называть POLER-OS Windows
Коммерческая тайна	Низкий	Использование leak-исходников	Только black-box анализ

4. Архитектура совместимости с Windows

4.1 PE Loader

PE (Portable Executable) Loader — компонент, загружающий .exe файлы. Должен: парсить DOS header -> PE header -> Optional Header -> Section Table; загружать секции в память с правильными адресами и правами; обрабатывать imports (IAT resolution); инициализировать TLS; передавать управление в Entry Point. Anti-cheat проверяет: совпадение загруженного образа с файлом на диске (хеш секций), целостность IAT, корректность PE->ImageBaseAddress. Сложность: 8/10, ~10-15 недель.

4.2 DLL Loader

DLL Search Order: Known DLLs (предзагруженные из System32), AppPath, SxS manifests (side-by-side assembly versioning), API Set mapping (api-ms-win-*.dll -> реальная DLL). Каждый DLL имеет собственный PE header, imports, TLS. Сложность: 9/10, ~15-20 недель.

4.3 Object Manager, Registry, Security Reference Monitor

NT Object namespace — иерархия объектов: Device, BaseNamedObjects, Registry, KernelObjects. Каждый объект имеет тип, security descriptor, reference count. Anti-cheat использует NtOpenDirectoryObject/NtQueryDirectoryObject для перечисления объектов. Сложность: 7/10, ~8-12 недель. Registry: достаточно ключей, читаемых античитами: HKLM_SYSTEM_CurrentControlSet_Services, HKLM_SOFTWARE_Microsoft_Windows_NT_CurrentVersion, HKLM_SOFTWARE_Microsoft_Cryptography. Сложность: 6/10, ~6-8 недель.

Token модель: Access Token (SID пользователя, группы, привилегии, integrity level), impersonation, SeDebugPrivilege, SeLoadDriverPrivilege. Integrity levels: Untrusted -> Low -> Medium -> High -> System. Сложность: 7/10, ~8-10 недель.

4.4 ALPC, Scheduler, Memory Manager

ALPC — основной IPC механизм Windows. Используется csrss.exe, сервисами, COM. Anti-cheat общается с сервисом через ALPC. Нужен: NtConnectPort, NtAlpcSendWaitReceivePort. Сложность: 7/10, ~10-16 недель. Scheduler: Thread priorities (0-31, real-time 16-31), priority boost для I/O, affinity mask. Сложность: 8/10, ~12-16 недель. Memory Manager: VAD tree, section objects, page fault handling, COW. Сложность: 9/10, ~16-20 недель.

4.5 Драйверная модель и системные вызовы

Невозможно загрузить .sys драйверы. Windows kernel drivers скомпилированы для ntoskrnl.exe. Решение: собственный драйверный формат (POLER Driver Interface), shim-слой для популярных IOCTL. Это исключает загрузку kernel-mode античитов — но контейнерная изоляция (раздел 5) решает эту проблему иначе. Системные вызовы: ntdll.dll shim перехватывает все syscall и маршрутизирует в POLER-OS native syscall table. Приложение вызывает Win32 API -> ntdll.dll shim -> POLER-OS kernel. Каждый процесс имеет РЕВ и ТЕВ. Структуры EPROCESS, ETHREAD должны быть реализованы с правильными offset-ами для NtQuerySystemInformation.

5. Античит-совместимость и контейнерная изоляция

5.1 Принцип: корректная реализация, не обход

POLER-OS не обманывает античиты — она предоставляет им то, что они ожидают найти: полностью реализованную Windows-совместимую среду, написанную с нуля. Разница между хаком и реализацией: хак — `if(anti_cheat_check) return TRUE` (обман); реализация — полноценный `NtQuerySystemInformation()`, который возвращает правильные данные, потому что система реально работает как Windows. Это фундаментальное различие: POLER-OS не имитирует Windows, она является Windows-подобной системой с нативной поддержкой NT API.

5.2 Что решается и что НЕ решается корректной реализацией NT API

Проверка античита	Что проверяет	Реализация POLER-OS	Статус
PEB/TEB integrity	Структуры процесса/потока	Полная реализация PEB/TEB	Решаемо
NtQuerySystemInfo	Список процессов, модулей	Native syscall через POLER	Решаемо
EPROCESS/ETHREAD	Kernel структуры	Полная реализация структур	Решаемо
Object Manager	Device, BaseNamedObjects	Полный NT Object namespace	Решаемо
Registry queries	Версия ОС, драйверы, GUID	Registry с правильными ключами	Решаемо
IAT integrity	Import Address Table	Корректный PE loader	Решаемо
Heap integrity	Консистентность кучи	NT heap allocator	Решаемо
File system checks	Пути, ACL, file attributes	POLER-FS с NT ACL	Решаемо
User-mode debugging	Отладочные API	Correct Dbg* API behaviour	Решаемо
Проблема	Причина		Тип ограничения
Kernel-mode .sys драйверы	Скомпилированы для ntoskrnl.exe		Техническое
WHQL подпись	Требуется Microsoft certification		Политика
VBS/HVCI	Требуется Hyper-V гипервизор		Техническое
TPM attestation	Microsoft PKI, не наш сертификат		Политика
ELAM	Требуется Microsoft-signed драйвер		Политика
Secure Boot chain	Microsoft UEFI CA		Политика

5.3 Совместимость по конкретным античитам

Античит	Тип	Шанс совместимости	Причина
VAC (Valve)	User-mode	Высокий	Только user-mode проверки
EQU8	Преим. user-mode	Средний	User-mode + опциональный kernel
EAC (EasyAntiCheat)	Kernel driver	Низкий	Требуется загрузка .sys драйвера
BattlEye	Kernel driver	Низкий	Агрессивные kernel проверки

Античит	Тип	Шанс совместимости	Причина
Vanguard (Riot)	Boot-start + HVCI	Очень низкий	Требует HVCI + TPM
HoYoverse (miHoYo)	User-mode + собств.	Средний-Высокий	Собственный античит, частично user-mode

Единственный реальный путь к поддержке kernel-mode античитов — сотрудничество с их разработчиками: портирование античита на POLER-OS kernel API, собственный Anti-Cheat SDK, сертификационная программа. Однако POLER-OS обладает мощным альтернативным механизмом — контейнерной изоляцией, которая делает многие функции kernel-античитов избыточными.

5.4 Модель изоляции процессов: контейнерная архитектура POLER-OS

Ключевой архитектурный принцип POLER-OS: в системе не существует понятия демона или вируса в традиционном смысле. Каждый процесс выполняется в строго изолированном контейнере. Один контейнер — одно программное обеспечение. Процессы из разных контейнеров не пересекаются.

Традиционные операционные системы (Windows, Linux) построены на модели, где процессы сосуществуют в общем адресном пространстве и могут взаимодействовать друг с другом через множество каналов: разделяемая память, отладочные API, инъекции DLL, сигналы, pipe-ы, сокеты. Это создаёт фундаментальную проблему: любой процесс потенциально может вмешаться в работу любого другого процесса. Именно эта архитектурная уязвимость делает необходимыми античиты, антивирусы, firewall-ы и другие защитные механизмы — они являются «заплатками» на модели, которая по своей природе позволяет нежелательное взаимодействие.

POLER-OS подходит к проблеме иначе: вместо того чтобы разрешать взаимодействие и затем бороться с его вредоносными формами, система делает взаимодействие между контейнерами невозможным на архитектурном уровне. Это не ограничение — это фундаментальное свойство системы, аналогичное тому, как физические контейнеры на корабле не позволяют грузу перемешиваться. Каждый контейнер — это изолированная программная среда, внутри которой работает ровно одно программное обеспечение со всеми его взаимодополняющими процессами.

Античит как гость, а не как хозяин

Античит в POLER-OS — это не привилегированный компонент системы, а гость, работающий внутри изолированного контейнера вместе с игрой. Он не имеет власти над ОС в целом, потому что контейнерная изоляция не позволяет ему выйти за пределы своего программного окружения. Античит полезен: он контролирует целостность исполняемого кода и обнаруживает модификации внутри своего контейнера. Но его работа ограничена границами этого контейнера, и вмешиваться в его работу нет необходимости — он выполняет свою функцию корректно.

Это принципиально отличается от модели Windows, где античит (особенно kernel-mode) является привилегированным компонентом с доступом ко всей системе. В Windows античиту нужен kernel-access, потому что без него любой внешний процесс может инъектировать код в процесс игры. В POLER-OS эта угроза устранена архитектурно — поэтому античит не нуждается в привилегиях суперпользователя.

Пример: Genshin Impact и античит HoYoverse

Рассмотрим конкретный сценарий. Genshin Impact использует собственную античит-систему HoYoverse. В POLER-OS Genshin Impact и HoYoverse античит — это одно программное обеспечение, работающее внутри одного контейнера. Система воспринимает их как единый процесс. Чит-инструменты вроде Kogeri — это совершенно другой процесс, который работает в другом контейнере. Поскольку процессы не пересекаются, Kogeri физически не может получить доступ к памяти или данным Genshin Impact, даже если бы хотел. Изоляция контейнера сама по себе делает невозможным внешнее вмешательство. Однако античит сохраняет свою функцию: он контролирует целостность кода внутри контейнера и может обнаруживать модификации исполняемых файлов.

Установка пиратского ПО

Если кто-то устанавливает пиратский софт с уже пропатченной игрой — это один пакет, одна программа в одном контейнере. Система этому не препятствует: пиратская копия — это целостный программный пакет, который запускается в своей изолированной среде. Однако если злоумышленник попытается установить чит-модуль поверх уже установленного оригинала — такого сделать уже не получится. Причина: в контейнере может работать только одно программное обеспечение. Это Genshin Impact и его взаимодополняющие процессы — для системы это один процесс. Рядом вмешаться в него нельзя, потому что для вмешательства нужен другой процесс, а другой процесс не может пересекаться с первым. Вмешаться в работу уже установленного оригинала извне — архитектурно невозможно.

Архитектурные преимущества контейнерной модели

Аспект	Традиционная модель (Windows/Linux)	Контейнерная модель POLER-OS
Взаимодействие процессов	Процессы могут читать/писать память друг друга (отладочные API, инъекции DLL)	Процессы полностью изолированы; чтение чужой памяти невозможно
Вирусы и трояны	Могут внедряться в работающие процессы	Невозможно внедриться; каждый контейнер изолирован
Античит	Нужен kernel-доступ для защиты от инъекций	Изоляция уже обеспечивает защиту; античит контролирует только целостность файлов
Читы (инъекция)	Внедряют DLL в процесс игры	Невозможно; другой процесс не может инжектировать код в контейнер
Читы (пропатченная игра)	Распространены	Работают как отдельный пакет; система не предотвращает, но оригинал защищён
Kernel-mode защита	Необходима (EAC, BattlEye)	Не нужна; изоляция контейнера заменяет kernel-level защиту

6. Собственная криптография: инструмент быстродействия и совместимости

6.1 Фундаментальный пересмотр: криптография — это не только шифрование

Собственная криптография в POLER-OS предназначена не для защиты данных от криптоаналитика, а для обеспечения быстродействия и совместимости при управлении операционной системой. Это ключевое различие, которое меняет всю оценку рисков и назначение крипто-инструментов.

Версии 1.0 и 2.0 данного аудита рассматривали собственную криптографию через призму классической криптографической модели: «защита данных от несанкционированного доступа». Версия 3.0 полностью пересматривает этот подход на основании уточнённого понимания архитектуры POLER-OS и роли криптографии в управлении операционной системой. Криптография — это не синоним шифрования. Это широкий спектр функций, каждая из которых имеет своё назначение и свои требования к производительности: шифрование (прячет содержание), аутентификация (доказывает кто), целостность (доказывает что не изменено), неотказуемость (нельзя отрицать действие), подтверждение источника (доказывает откуда). Собственная криптография POLER-OS работает именно в последних четырёх функциях, а не в первой.

6.2 Криптология vs бинарная логика: аналогия с клавиатурой

Чтобы понять назначение собственной криптографии POLER-OS, рассмотрим фундаментальную разницу между двумя подходами к обработке сигналов в компьютерных системах.

Бинарная логика (без криптографии)

Когда пользователь нажимает клавишу W на клавиатуре в традиционной системе, происходит следующее: клавиатура отправляет скан-код 0x11, операционная система получает число 0x11 и интерпретирует его как «W нажата». Всё. ОС не знает и не может знать: нажал ли человек пальцем, или SendInput() вызвал скрипт, или драйвер подсунул поддельный скан-код, или чтение из /dev/input подделал rootkit. Для ОС это просто бит. Любой источник неотличим от человека. Это и есть дыра — окно для инъекций, автоматизации, читов, малвари. В бинарной логике сигнал не имеет удостоверения личности.

Криптология (с криптографией)

Когда пользователь нажимает клавишу W в системе с криптографической верификацией, происходит следующее: клавиатура (с аппаратным токеном/TPM) отправляет скан-код 0x11 + HMAC(0x11, timestamp, hardware_key), ОС получает не просто число, а доказательство: HMAC — это подпись с ключом, который только у аппаратуры; timestamp — защита от повтора (replay attack); ОС верифицирует — если подпись не сходится, значит это программная инъекция. Разница фундаментальная: ОС знает ОТКУДА пришёл сигнал. Криптография здесь не прячет данные — она подтверждает источник.

Аспект	Бинарная логика	Криптология
Сигнал	Просто число (0x11)	Число + подпись (0x11 + HMAC)
Источник	Неизвестен (любой)	Верифицирован (железо/человек)
Подделка	Тривиальна (SendInput)	Невозможна без ключа
Replay attack	Возможна	Защищена (timestamp)

Аспект	Бинарная логика	Криптологика
Назначение крипто	---	Не шифрование, а аутентификация

6.3 Почему RSA не подходит для управления ОС

RSA спроектирован для защиты коммуникаций — асимметричное шифрование с открытым ключом, где секретность базируется на задаче факторизации больших чисел. Для этой задачи RSA исторически подходил. Однако для управления операционной системой RSA категорически не подходит по нескольким причинам.

Во-первых, производительность: RSA-2048 требует ~1 мс на операцию подписи и ~0.05 мс на верификацию. Для ОС, где криптография встроена в каждый syscall, каждый драйвер и каждую файловую операцию, это неприемлемо. Представьте: каждое нажатие клавиши, каждый запрос к драйверу, каждое обращение к ФС — каждый из этих вызовов должен быть верифицирован. RSA добавил бы секунды задержки на каждую операцию, что делает систему неработоспособной.

Во-вторых, криптоаналитическая стойкость: RSA взламывается на любом компьютере за несколько секунд при современном уровне вычислительных возможностей, независимо от количества бит в ключе. Это не уязвимость реализации — это фундаментальное свойство алгоритма, который основан на задаче факторизации. Квантовые вычисления (алгоритм Шора) делают RSA полностью непригодным. Но даже без квантовых компьютеров прогресс в классическом факторинге сокращает время взлома экспоненциально.

Windows Win32 API, Linux API и другие операционные системы используют встроенную криптографию для управления — и POLER-OS должна делать то же самое, но с алгоритмом, который подходит именно для этой задачи: быстрый, лёгкий, встроенный в ядро, способный верифицировать тысячи операций в секунду без задержек.

6.4 Назначение собственной криптографии в POLER-OS

Собственная криптография POLER-OS (PND v8, POLER-CTR) используется как механизм управления ОС — аналог того, как Win32 API использует свои внутренние структуры, или как Linux API использует ioctl. Это не «замок на двери», а «паспорт на КПП»: она не прячет, она подтверждает, кто ты и откуда пришёл, и делает это быстро, потому что ядро не может ждать миллисекунды на RSA-подпись для каждого прерывания.

Область применения	Назначение	Требования	Почему стандартные алгоритмы не подходят
Управление драйверами	Верификация целостности и подлинности драйверов	Скорость проверки при каждой загрузке; малый оверхед на syscall	RSA: слишком медленный; AES-GCM: не обеспечивает асимметричную верификацию
Файловая система	Контроль целостности метаданных и данных	O(1) проверка per-block; интеграция с NVMe pipeline	SHA-256: 3-5x медленнее BLAKE3; RSA-подпись на блок невозможна
Межпроцессное взаимодействие	Аутентификация сообщений между контейнерами	Низкая латентность; невозможность подделки	TLS добавляет неприемлемую латентность для IPC
Реестр (Registry)	Целостность hive-файлов	Быстрая проверка при каждом запросе	RSA: недопустимо медленный для per-query верификации

Область применения	Назначение	Требования	Почему стандартные алгоритмы не подходят
Загрузка ОС	Верификация цепочки загрузки	Скорость проверки <100ms для всего boot-процесса	RSA верификация на каждом этапе добавляет секунды задержки

6.5 Связь с DirectInput и DLC

Аналогия с клавиатурным вводом напрямую связана с протоколами DirectInput/DirectX. DirectInput (и современные XInput, Raw Input) — это попытка приблизиться к криптологике на уровне API: Raw Input получает данные напрямую от устройства, минуя оконные сообщения (WM_KEYDOWN), которые любой процесс может подделать через SendMessage; XInput работает с контроллерами через аутентифицированный протокол. Но даже это — не полная криптография, а просто более короткий путь от железа к приложению. Чит-программы как раз эксплуатируют разрыв между «бинарная логика думает, что это человек» и «на самом деле это софт».

DLC (Digital Rights Management) использует ту же модель: без криптографии файл dlc_unlock.bin лежит на диске, и любой может его создать или скопировать; с криптографией файл + цифровая подпись издателя, подделать подпись практически невозможно. ОС проверяет «файл подписан издателем?» вместо «файл существует?». POLER-OS использует аналогичный подход на уровне управления ОС: драйвер верифицирован — он загружен; IPC-сообщение подписано — оно обработано; файл целостен — он доступен.

6.6 Разделение ролей: внутренняя криптография vs защита данных

Собственная криптография POLER-OS не заменяет стандартные алгоритмы там, где они уместны. Для защиты данных пользователя (шифрование диска, VPN, TLS) используются проверенные стандарты: AES-256-XTS для дискового шифрования (per-volume), AES-256-GCM для новых файлов (per-inode), ChaCha20-Poly1305 как fallback при отсутствии AES-NI. Собственная криптография используется исключительно для внутренних механизмов ОС: верификация драйверов, контроль целостности ФС на уровне блоков, аутентификация IPC-сообщений между контейнерами, верификация boot-цепочки. Это разделение аналогично тому, как Windows использует свои внутренние криптографические примитивы для управления ОС, отличные от тех, что предоставляются через CryptoAPI для пользовательских приложений.

6.7 Безопасность собственной криптографии: переоценка рисков

Утверждение, что собственная криптография без peer review опасна, справедливо только если криптография используется для защиты данных. Но когда криптография является инструментом быстродействия и совместимости, модель угроз принципиально иная. Если алгоритм взломан, последствия — не утечка данных (данные защищены AES-GCM), не компрометация коммуникаций (они защищены TLS), а снижение производительности — система переключается на стандартный алгоритм с более высокой латентностью. Это acceptable risk.

Собственная криптография в POLER-OS — это оптимизация, а не единственный рубеж защиты. Если она не справляется, система gracefully деградирует до стандартных алгоритмов, теряя в скорости, но не в безопасности. Это аналогично тому, как кэш процессора является оптимизацией: если кэш-промах —

система не падает, а просто обращается к RAM медленнее. Никакие пользовательские данные не подвергаются риску, потому что они защищены отдельным слоем стандартных алгоритмов шифрования.

7. Roadmap развития проекта

Полная дорожная карта от текущего состояния до релиза 1.0, с оценками для одного разработчика.

Фаза	Описание	Приор	Сложн	Трудоём.	Риски
1. Минимальное ядро	Boot, GDT, IDT, paging	P0	6	8-12 нед.	Аппаратные баги QEMU
2. Управление памятью	PMM, VMM, heap, slab	P0	7	10-14 нед.	Фрагментация, leaks
3. SMP	APIC, IPI, TLB shutdown	P0	9	12-16 нед.	Data corruption, deadlocks
4. Планировщик	Потоки, процессы, приоритеты	P0	7	8-12 нед.	Priority inversion
5. Синхронизация	Spinlocks, mutexes, RCU	P0	6	6-8 нед.	Deadlocks, starvation
6. Файловые системы	VFS, ext2 (r/w)	P1	7	8-10 нед.	Data corruption
7. Драйверная модель	PCI, USB, AHCI	P1	8	10-14 нед.	Железо-специфичные баги
8. Сеть	TCP/IP, sockets	P1	8	12-16 нед.	Стек-уязвимости
9. Графика	Framebuffer, compositor	P2	7	8-10 нед.	GPU-specific
10. Vulkan GPU	ICD driver	P1	10	30-40+ нед.	CRITICAL
11. User space	ELF loader, libc, shell	P1	7	10-14 нед.	libc completeness
12. VFS complete	POLER-FS basic	P1	8	12-16 нед.	Формат стабильность
13. Win32 subsystem	PE/DLL loader, basic Win32	P1	9	20-30 нед.	CRITICAL: огромный API
14. NT API	ntdll, ObjMgr, Registry, SRM	P1	10	30-40+ нед.	CRITICAL: ~2000 функций
15. App compat	Первые Windows-приложения	P2	8	12-20 нед.	Edge cases
16. Контейнерная изоляция	Изоляция процессов, IPC	P1	8	10-14 нед.	Совместимость

Итого без упрощений: ~252 недели (4.8 года). С упрощениями (Mesa3D fork, userspace shim, ext4+encryption, lwIP, musl, контейнерная изоляция) — экономия ~108 недель, итог: ~3-4 года до beta.

8. Критический аудит проекта

8.1 Слабые места

#	Слабое место	Влияние	Серьёзность
1	TLB shutdown отложен	SMP data corruption гарантирован	CRITICAL
2	MAX_TASKS=8	Неприемлемо для реальной нагрузки	CRITICAL
3	Нет AHCI/NVMe драйвера	OS работает только в QEMU	CRITICAL
4	Shim vs kernel-native не решено	Архитектурное противоречие	HIGH
5	Immutable kernel + anti-cheat	Решается контейнерной изоляцией (раздел 5.4)	HIGH -> LOW*
6	Собственная криптография	Инструмент быстрогодействия, не защиты (раздел 6)	HIGH -> LOW*
7	Один разработчик	Single point of failure	MEDIUM
8	Нет автоматического тестирования	Регрессии не детектируются	MEDIUM

**Понижено в v3.0: контейнерная изоляция решает проблему античитов; собственная криптография пересмотрена как инструмент быстрогодействия.*

8.2 Возможные архитектурные ошибки

1. Попытка быть полноценным Windows вместо достаточного Windows. ReactOS потратила 20+ лет, пытаясь клонировать Windows полностью, и всё ещё не готова. POLER-OS должна сфокусироваться на конкретных сценариях (игры, офисные приложения) и реализовать только необходимый минимум API для них.
2. Kernel-native Win32 вместо userspace shim. Реализация Win32 в ядре создаёт огромную поверхность атаки и требует в 3-5 раз больше работы. Userspace shim (как в Wine) — проще, безопаснее, быстрее в разработке. Компромисс: shim для большинства функций, kernel-native только для критичных path.
3. Собственная криптография без чёткого разделения ролей (ИСПРАВЛЕНО В v3.0). Собственная криптография используется как инструмент быстрогодействия и совместимости для внутренних механизмов ОС. Для защиты данных используются стандартные алгоритмы (AES-256-XTS, AES-256-GCM, ChaCha20-Poly1305). Это разделение аналогично модели Windows, где внутренние криптографические примитивы отличаются от пользовательских CryptoAPI.

8.3 Альтернативные решения

Текущий план	Альтернатива	Преимущество	Экономия
Свой Vulkan driver	Mesa3D fork	Battle-tested code, community	~20 нед.
Kernel-native Win32	Userspace shim (Wine-like)	Проще, безопаснее, быстрее	~30 нед.
POLER-FS с нуля	ext4 + encryption layer	Стабильный формат	~10 нед.
Свой TCP/IP стек	lwIP port	Минимальный, проверенный	~10 нед.
Свой libc	musl port	Стандартный, полный	~6 нед.
PND + POLER-CTR (только защита)	Собств. для быстрогодействия + AES-GCM для защиты	Разделение ролей (v3.0)	~4 нед.

8.4 Масштаб проекта и реалистичные компоненты

Для сравнения: Linux kernel — ~30М строк кода, 1700+ разработчиков; ReactOS — ~12М строк, 20+ лет; FreeBSD — ~14М строк. POLER-OS для минимальной работоспособности (запуск notepad.exe) — ~500К-1М строк кода. Для игровой совместимости — ~3-5М строк. Реалистично для 3-5 разработчиков за 4-6 лет.

Компонент	Реалистичность	Время	Условие
Ядро + SMP + память	Реалистично	1-2 года	Один разработчик
VFS + ext2	Реалистично	0.5-1 год	
PE/DLL Loader	Реалистично	0.5-1 год	
Basic Win32	Реалистично	1-2 года	
Контейнерная изоляция	Реалистично	0.5-1 год	Интеграция в scheduler
Vulkan GPU driver	Многолетний	3-5 лет	Без Mesa3D fork
Полный NT API	Многолетний	5-8 лет	
Anti-cheat совместимость	Требует кооперации	Зависит	Сотрудничество с EAC/BE
Собственный POLER-FS	Многолетний	2-3 года	Полная реализация

8.5 Критические упрощения без потери архитектурной идеи

1. Mesa3D fork вместо своего Vulkan. Vulkan specification — 1800+ страниц. Fork Mesa3D (MIT-licensed) — 6-12 месяцев вместо 3-5 лет. Архитектурная идея (Vulkan-first GPU) сохраняется.
2. Userspace shim вместо kernel-native Win32. Wine/Proton доказали, что userspace shim работает для тысяч приложений. POLER-OS может использовать аналогичный подход с собственным ядром.
3. ext4 + encryption layer вместо POLER-FS с нуля. ext4 — battle-tested. Добавление encryption layer + NT ACL translation в VFS — 3-6 месяцев вместо 2-3 лет. POLER-FS может быть добавлена позже.
4. Контейнерная изоляция вместо kernel-античитов. POLER-OS использует контейнерную модель, которая делает инъекции невозможными на архитектурном уровне. Это не обход античитов — это более фундаментальное решение проблемы, которое античиты пытаются решить.

Первая цель: запустить notepad.exe. Не Vulkan, не Genshin, не античиты. Самое простое Windows-приложение. Если notepad.exe работает — архитектура верна.

9. Заключение

Проведённый технический аудит подтверждает, что архитектурная концепция POLER-OS жизнеспособна, но требует значительных корректировок в реализации. Основные выводы:

Файловая система POLER-FS требует перехода от block groups к extent-based архитектуре с B+ деревом; увеличения inode до 512 байт; добавления inline data, ADS, reparse points, oplocks; реализации per-file CoW; использования AES-256-XTS для дискового шифрования; BLAKE3 для integrity.

Юридическая сторона проекта законна: реализация Windows API через clean-room разработку защищена прецедентами Oracle v. Google, Lotus v. Borland, и Директивой EC 2009/24/EC.

Совместимость с Windows реализуема для user-mode приложений. Kernel-mode драйверы не могут быть загружены — но контейнерная модель изоляции процессов делает ненужными многие функции kernel-античитов. Процессы в POLER-OS изолированы на архитектурном уровне: инъекции DLL, чтение чужой памяти и вмешательство в работу других процессов невозможны. Античит является гостем в изолированном контейнере, а не привилегированным компонентом системы.

Собственная криптография (PND v8, POLER-CTR) пересмотрена в v3.0: она является инструментом быстрого действия и совместимости для внутренних механизмов ОС, а не инструментом защиты данных. Это аналогично тому, как криптографическая верификация клавиатурного ввода подтверждает, что клавишу нажал человек, а не скрипт — она не шифрует нажатие, а аутентифицирует его источник. Для защиты данных используются стандартные алгоритмы. Если собственная криптография не справляется, система gracefully деградирует до стандартных алгоритмов, теряя в скорости, но не в безопасности.

Реалистичный timeline: 3-4 года до работающей beta с предлагаемыми упрощениями (Mesa3D fork, userspace shim, ext4+encryption, lwIP, musl, контейнерная изоляция). Без упрощений — проект не доживёт из-за масштаба. Первая цель: запустить notepad.exe.